



CS231. Nhập môn Thị giác máy tính

Đặc trưng cạnh (edge)



Mục tiêu

- Hiểu khái niệm về cạnh trong ảnh và vai trò của cạnh trong việc mô tả nội dung hình ảnh.
- Nắm vững các phương pháp phát hiện cạnh phổ biến (Sobel, Prewitt, Canny).
- Hiểu cách biểu diễn đặc trưng ảnh dựa trên thông tin cạnh.
- Xây dựng các vector đặc trưng dựa trên thông tin cạnh để phục vụ các bài toán thị giác máy tính.



Khái niệm

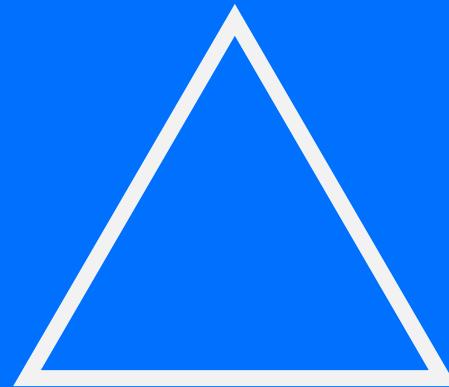


Dẫn nhập



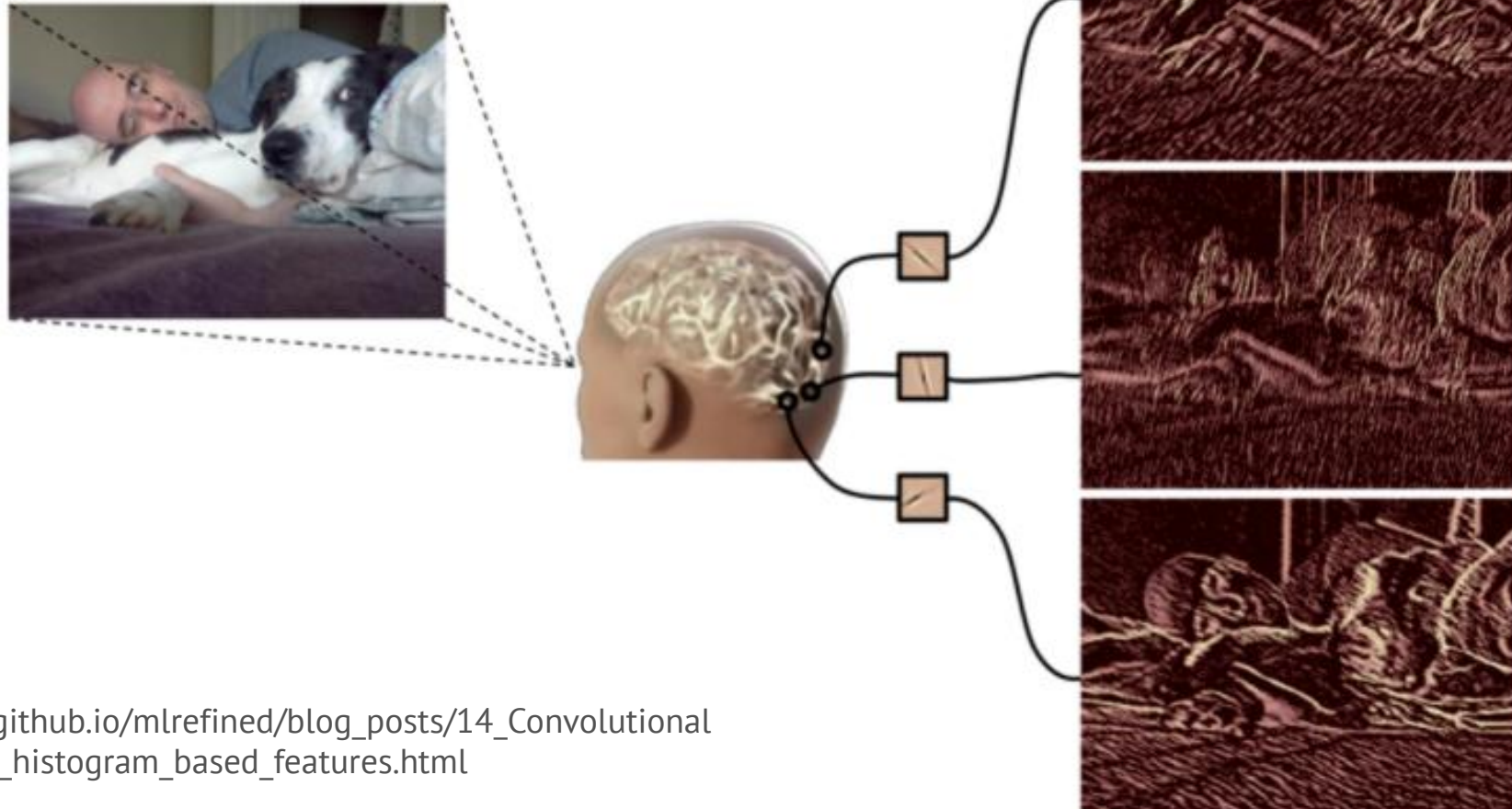


Dẫn nhập



Dẫn nhập

Cạnh (đường viền) ảnh là một đặc trưng rất quan trọng do **hệ thống thị giác của con người dựa trên cạnh để nhận thức** được đối tượng.



https://kenndanielso.github.io/mlrefined/blog_posts/14_Convolutional_networks/14_2_Edge_histogram_based_features.html



Định nghĩa

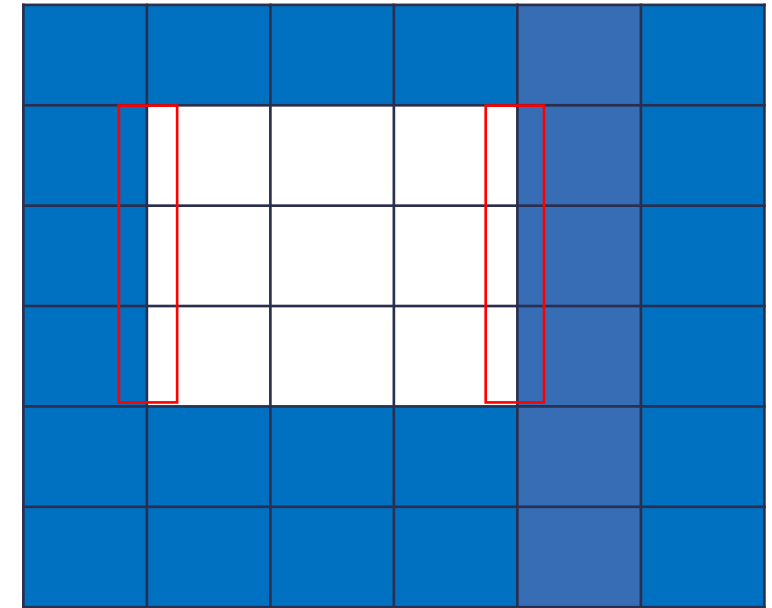
- Trong ảnh số, "**cạnh**" (edge) được định nghĩa là **sự thay đổi đột ngột về cường độ sáng của các pixel lân cận.**

Định nghĩa

Nếu chúng ta dựa trên:

1. Sự thay đổi cường độ sáng:

- **Cạnh** thường xuất hiện tại ranh giới **giữa hai vùng có độ sáng khác nhau**. Ví dụ, ranh giới giữa một vật thể sáng và nền tối.
- Sự thay đổi này có thể là sự tăng hoặc giảm đột ngột về độ sáng.



Định nghĩa

Nếu chúng ta dựa trên:

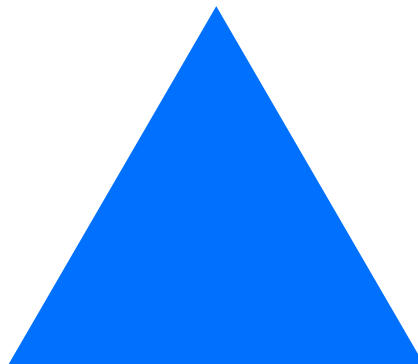
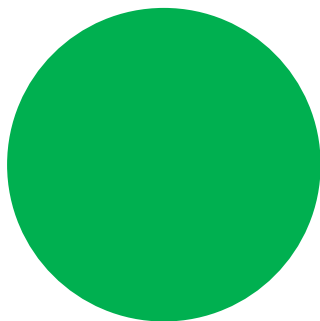
2. Pixel lân cận:

- **Cạnh** được xác định bằng cách **so sánh cường độ sáng** của một pixel với các pixel xung quanh nó.
- Nếu có sự khác biệt đáng kể, pixel đó được coi là một phần của cạnh.

0	0	0	0	0	0
0	255	255	255	0	0
0	255	255	255	0	0
0	255	255	255	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Tầm quan trọng

- Cung cấp thông tin về hình dạng và cấu trúc của các đối tượng trong ảnh.
- Ít bị ảnh hưởng bởi sự thay đổi về ánh sáng hoặc màu sắc.



Các ứng dụng chính

- Nhận dạng đối tượng,
- Phát hiện biên,
- Phân đoạn ảnh,
- Theo dõi đối tượng

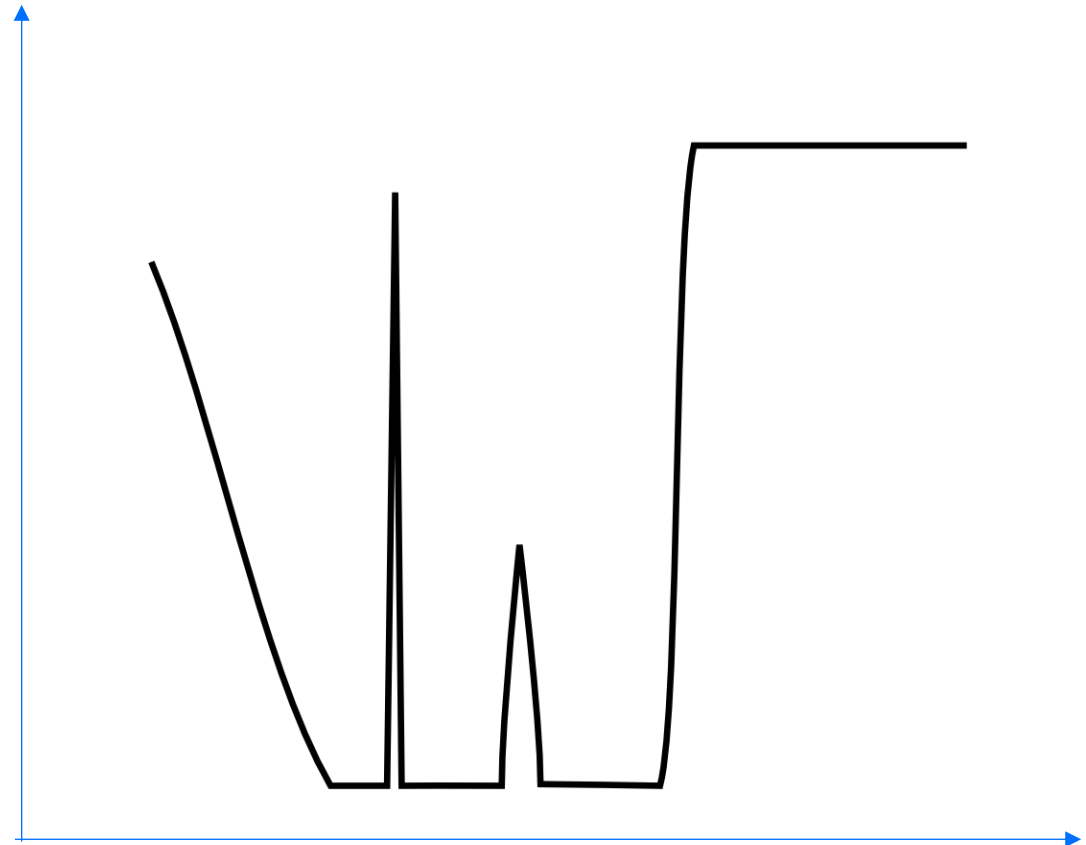
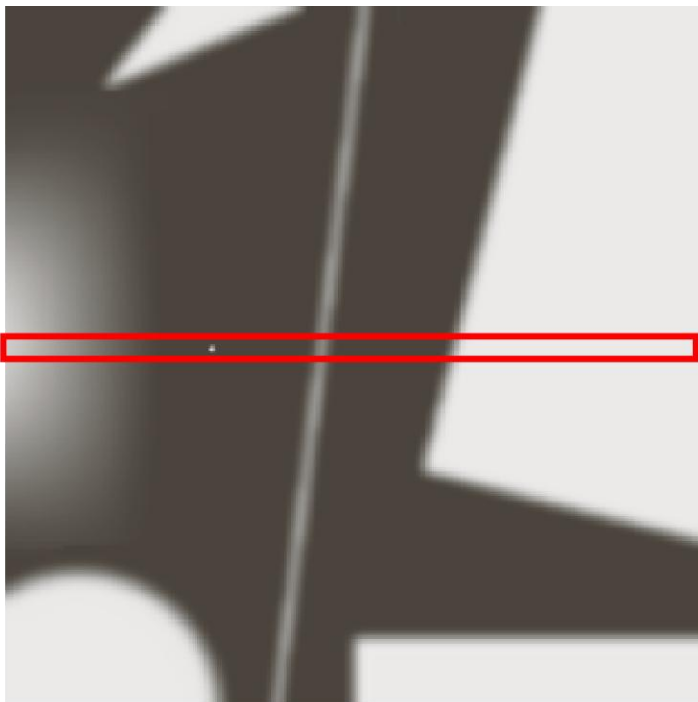




Dò tìm cạnh trong ảnh

Dò tìm cạnh trong ảnh

- Việc dò tìm cạnh ảnh có thể dựa vào độ thay đổi của các giá trị pixel.



Dò tìm cạnh trong ảnh

Xác định ranh giới
giữa hai vùng có độ
sáng khác nhau ?



$$\nabla f$$

Gradient



Gradient là gì ?

Gradient của hàm $f(v)$ với $v = (v_1, v_2, \dots, v_n)$ là một vector:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial v_1} \\ \frac{\partial f}{\partial v_2} \\ \vdots \\ \frac{\partial f}{\partial v_n} \end{bmatrix}$$

Mỗi thành phần trong vector đó là một đạo hàm riêng phần (*partial derivative*) theo từng biến của hàm đó



Gradient là gì ?

Ví dụ: Gradient của hàm $f(v) = 3x^2 + 2y + 5$ là một vector:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 6x \\ 2 \end{bmatrix}$$

Tính gradient cho ảnh số ?





Tính gradient cho ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	0	0	0	0	0
	1	0	255	255	255	0	0
	2	0	255	255	255	0	0
	3	0	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Ảnh số

$f(x,y)$

Hàm số theo hai biến số nguyên x, y .

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Gradient



Tính gradient cho ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	0	0	0	0	0
	1	0	255	255	255	0	0
	2	0	255	255	255	0	0
	3	0	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Ảnh số

$f(x=3, y=2)$

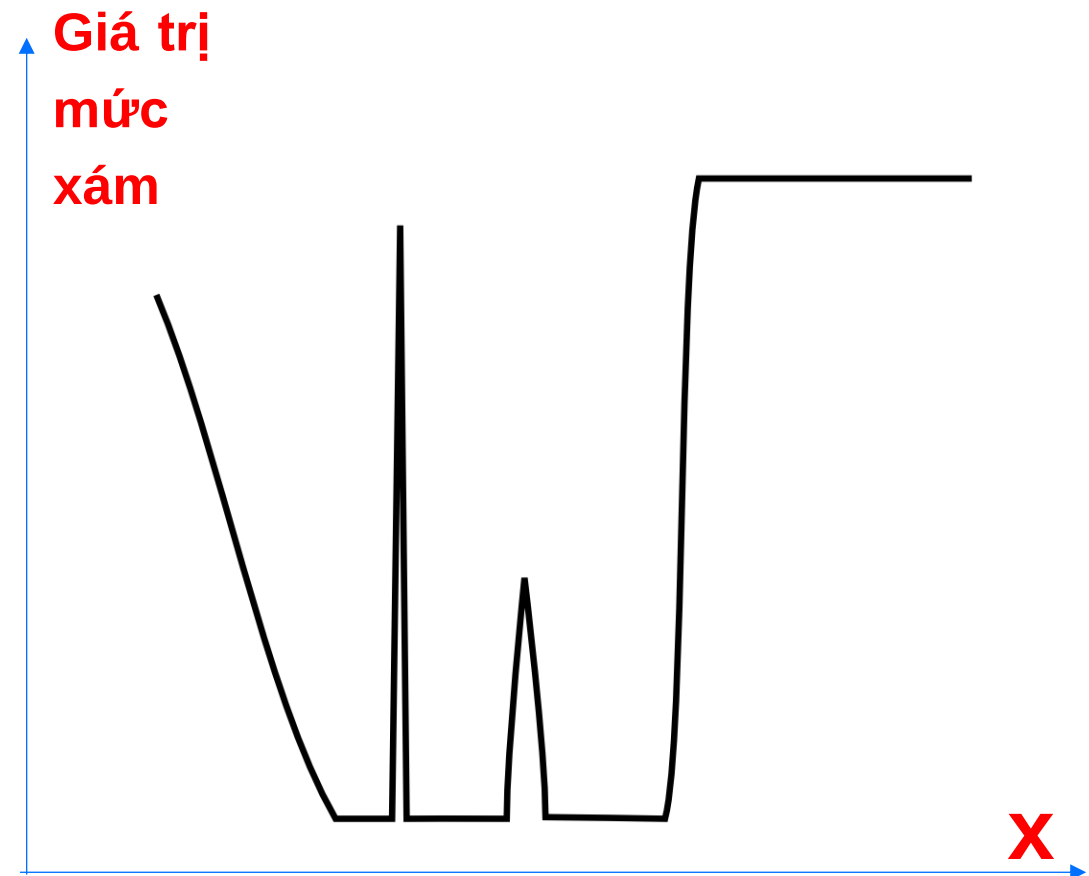
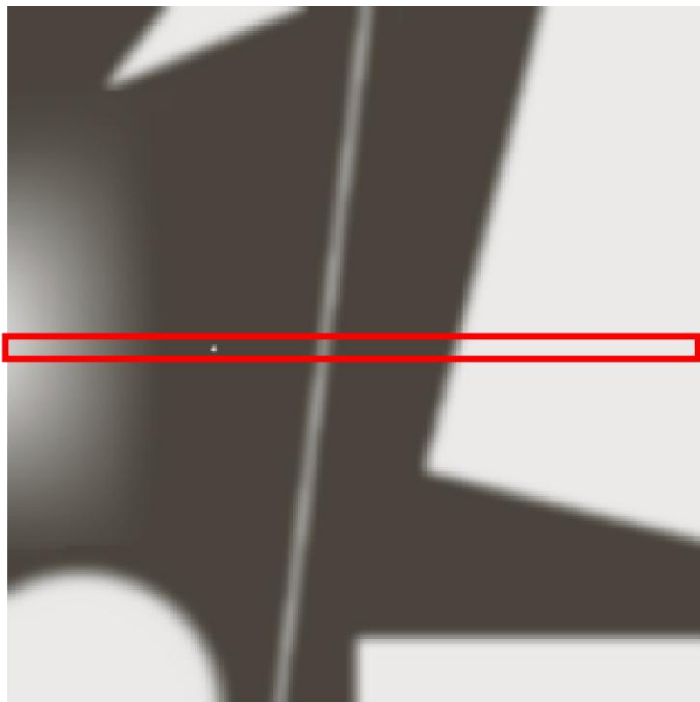
		j	$j + 1$
i	i		
	$i + 1$		

$$\nabla f_{j,i} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} \approx \begin{bmatrix} f(x = j + 1, y = i) - f(x = j, y = i) \\ f(x = j, y = i) - f(x = j, y = i + 1) \end{bmatrix}$$

Gradient

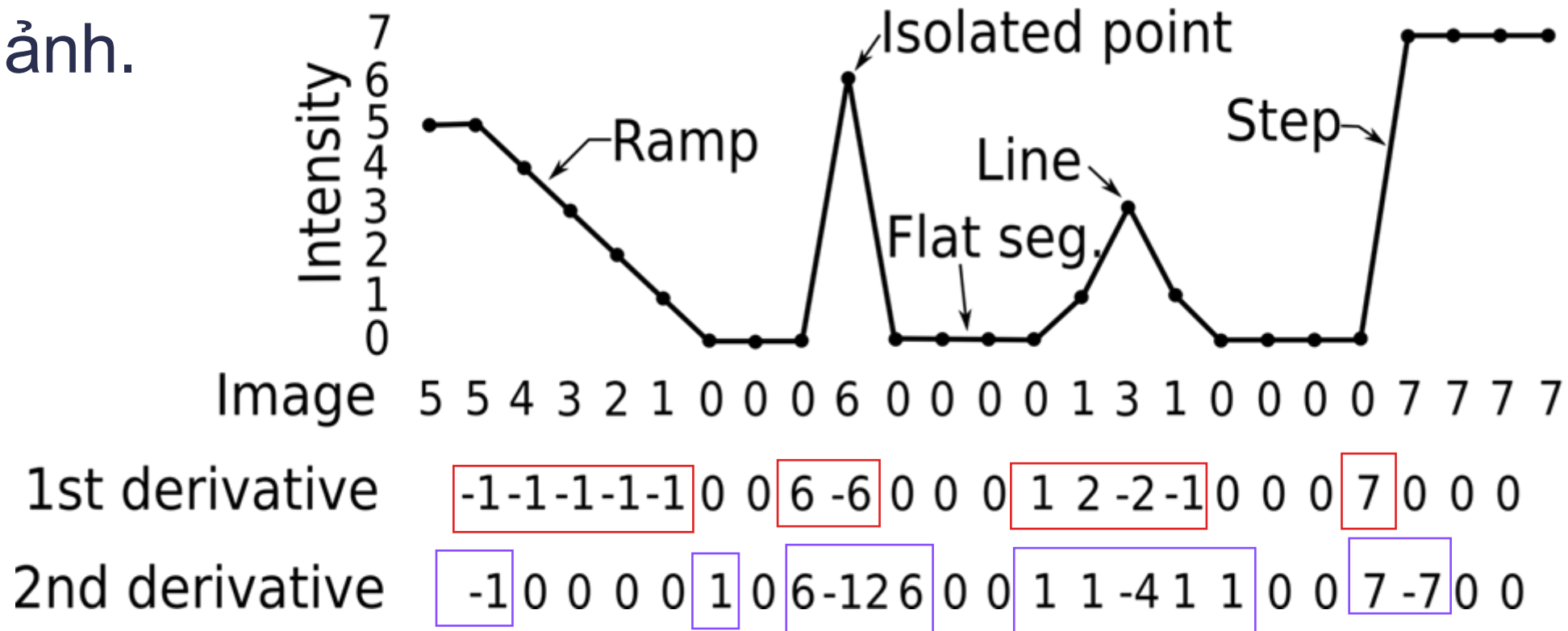
Tính gradient cho ảnh số

- Ví dụ: Xét độ thay đổi của các giá trị pixel của 1 dòng ảnh.



Tính gradient cho ảnh số

- Ví dụ: Xét độ thay đổi của các giá trị pixel của 1 dòng ảnh.



https://miac.unibas.ch/SIP/07-Segmentation-media/figs/1st_vs_2nd_derivative.png



Tính gradient cho ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	0	0	0	0	0
	1	0	255	255	255	0	0
	2	0	255	255	255	0	0
	3	0	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	0	0	0	0	0
	1	255	0	0	-255	0	0
	2	255	0	0	-255	0	0
	3	255	0	0	-255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

G_x = Ảnh sau khi tính Gradient theo x



Tính gradient cho ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	0	0	0	0	0
	1	0	255	255	255	0	0
	2	0	255	255	255	0	0
	3	0	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

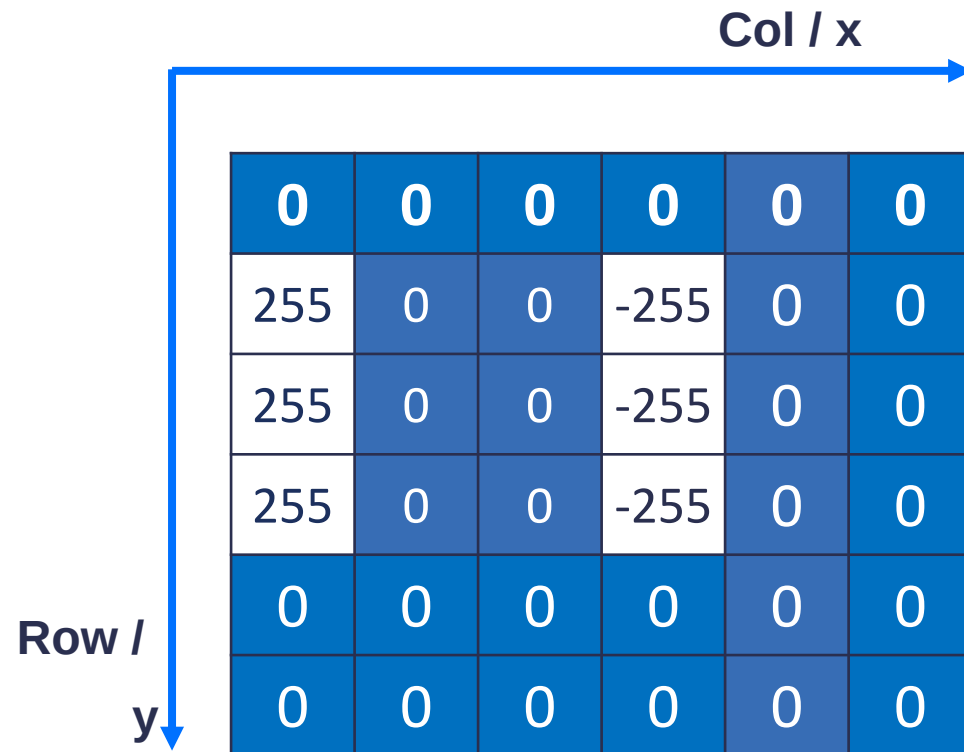
Ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	-255	-255	-255	0	0
	1	0	0	0	0	0	0
	2	0	0	0	0	0	0
	3	0	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

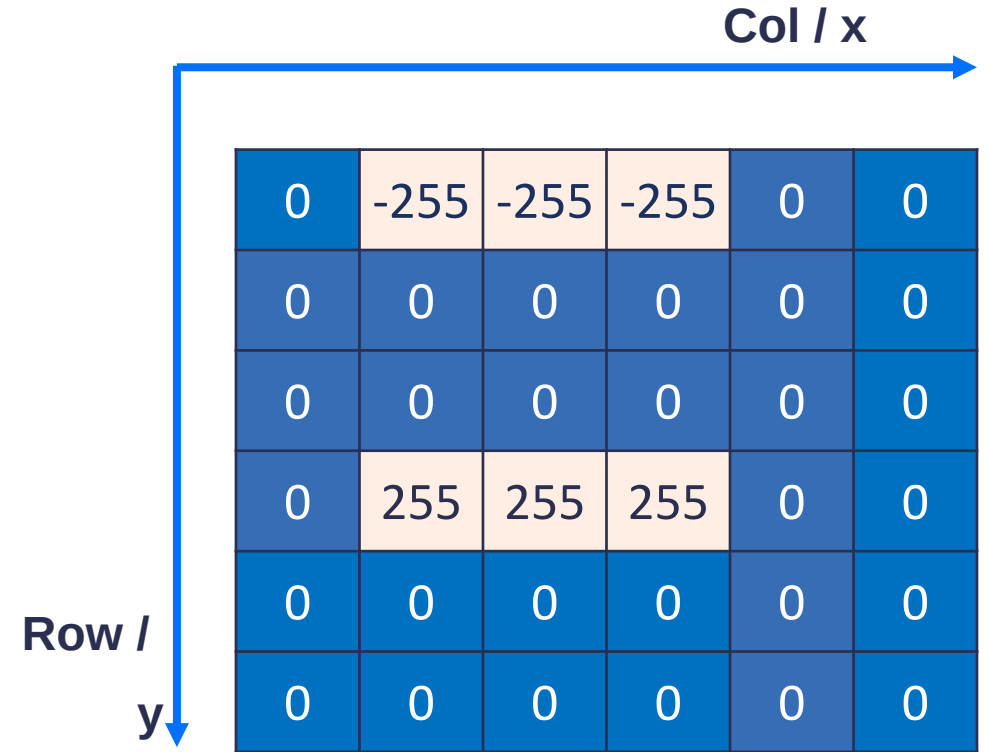
Gy = Ảnh sau khi tính Gradient theo y



Tính gradient cho ảnh số



Gx = Ảnh sau khi tính Gradient theo x



Gy = Ảnh sau khi tính Gradient theo y



Tính gradient cho ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	0	0	0	0	0
	1	0	255	255	255	0	0
	2	0	255	255	255	0	0
	3	0	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Ảnh số

		Col / x					
		0	1	2	3	4	5
Row / y	0	0	255	255	255	0	0
	1	255	0	0	255	0	0
	2	255	0	0	255	0	0
	3	255	255	255	255	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Ảnh sau khi tính Gradient theo x và y



Dò tìm cạnh trong ảnh

Các phương pháp cơ bản



Các phương pháp cơ bản

- First Order Derivative / Gradient Methods
 - Roberts Operator
 - Prewitt Operator
 - Sobel Operator
- Second Order Derivative
 - Laplacian
 - Laplacian of Gaussian
- Optimal Edge Detection
 - Canny edge detection

Toán tử Roberts

- Xét vùng ảnh như sau:

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

- Khi đó, ta có thể xấp xỉ:

$$|\nabla f(x, y)| \approx [(z_5 - z_8)^2 + (z_5 - z_6)^2]^{1/2}$$

- Khi cài đặt, ta có thể dùng các nhân tương ứng:

$$h_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \text{ and } h_2 = [1 \quad -1]$$

Toán tử Roberts

- Ngoài ra, ta có thể xấp xỉ:

$$|\nabla f(x, y)| \approx \left[(z_5 - z_9)^2 + (z_6 - z_8)^2 \right]^{1/2}$$

- Khi cài đặt, ta có thể dùng các nhân tương ứng:

$$h_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \text{ and } h_2 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Toán tử Roberts

- Các nhân tích chập trên còn được gọi là toán tử **Roberts cross-gradient operators**



Ảnh gốc



$$h_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \text{ and } h_2 = \begin{bmatrix} 1 & -1 \end{bmatrix}$$



$$h_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \text{ and } h_2 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$



Ví dụ

0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	255	255	255
0	0	0	0	0	0
0	0	0	0	0	0

Toán tử Prewitt

- Một xấp xỉ tốt hơn giá trị của gradient có thể được tính như sau:

$$|\nabla f(x, y)| \approx \left[((z_7 + z_8 + z_9) - (z_1 + z_2 + z_3))^2 + ((z_3 + z_6 + z_9) - (z_1 + z_4 + z_7))^2 \right]^{1/2}$$

- Tương ứng với nhân tích chập sau:

$$h_1 = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \text{ and } h_2 = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Toán tử Prewitt



Ảnh gốc



Prewitt

Toán tử Sobel

- Sử dụng 2 nhân tích chập sau:

$$H = \begin{array}{|c|c|c|} \hline -1 & -2 & -1 \\ \hline 0 & 0 & 0 \\ \hline 1 & 2 & 1 \\ \hline \end{array}$$

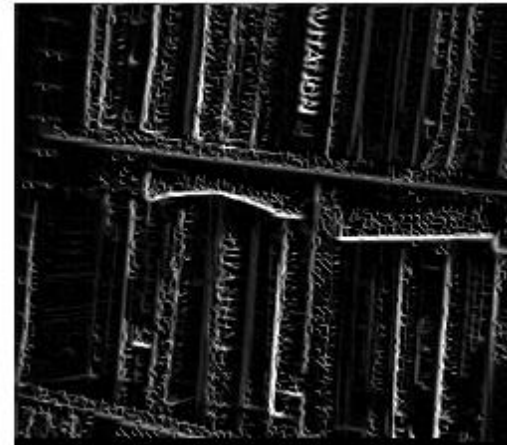
G
y

$$V = \begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -2 & 0 & 2 \\ \hline -1 & 0 & 1 \\ \hline \end{array}$$

G
x

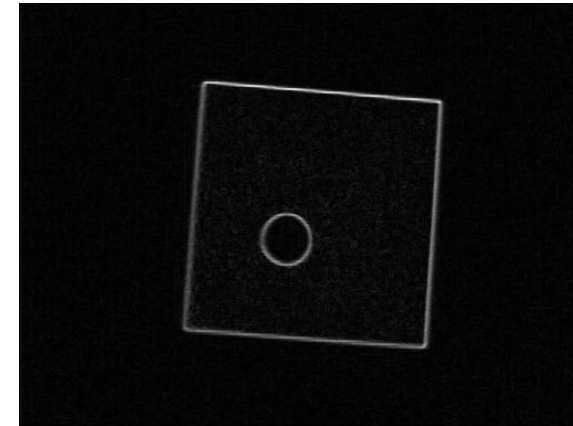
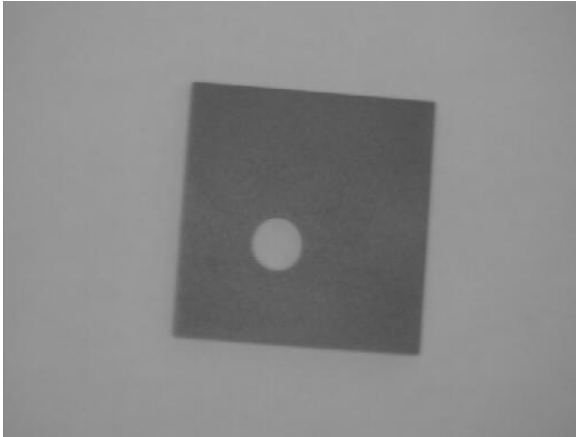
Toán tử Sobel

- Ví dụ:



- → Hạn chế của phương pháp này: nhạy cảm đối với nhiễu (nếu nhiễu lớn thì nó sẽ lấy nhiễu làm cạnh).

Toán tử Sobel



Ảnh gốc

Sobel

Toán tử Sobel



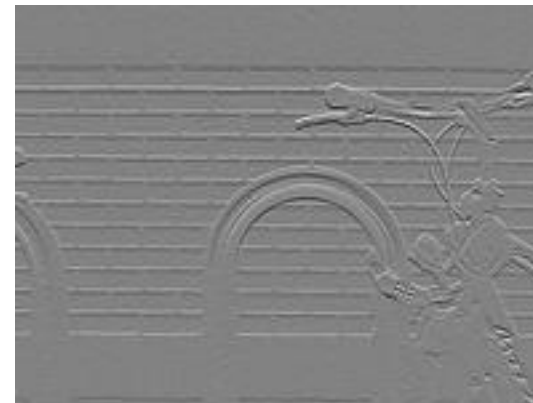
Grayscale image of a brick wall & a bike rack



Normalized sobel gradient image of bricks & bike rack



Normalized sobel x-gradient image



Normalized sobel y-gradient image

Toán tử Laplacian

- Tính đạo hàm bậc hai
- Hệ số nhân tích chập

$$L(x, y) = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

0	1	0
1	-4	1
0	1	0



Code minh họa

```
from scipy import ndimage

def sobel_filters(img):
    Kx = np.array([[ -1,  0,  1], [-2,  0,  2], [-1,  0,  1]], np.float32)
    Ky = np.array([[ 1,  2,  1], [ 0,  0,  0], [-1, -2, -1]], np.float32)

    Ix = ndimage.filters.convolve(img, Kx)
    Iy = ndimage.filters.convolve(img, Ky)

    G = np.hypot(Ix, Iy)
    G = G / G.max() * 255
    theta = np.arctan2(Iy, Ix)

    return (G, theta)
```



Code minh họa

```
kernels = np.array([ [-1, 0, 1],  
                     [-2, 0, 2],  
                     [-1, 0, 1]  
                   ]  
  
img_rst = cv2.filter2D(img_src, -1, kernels)
```





Trích xuất đặc trưng cạnh

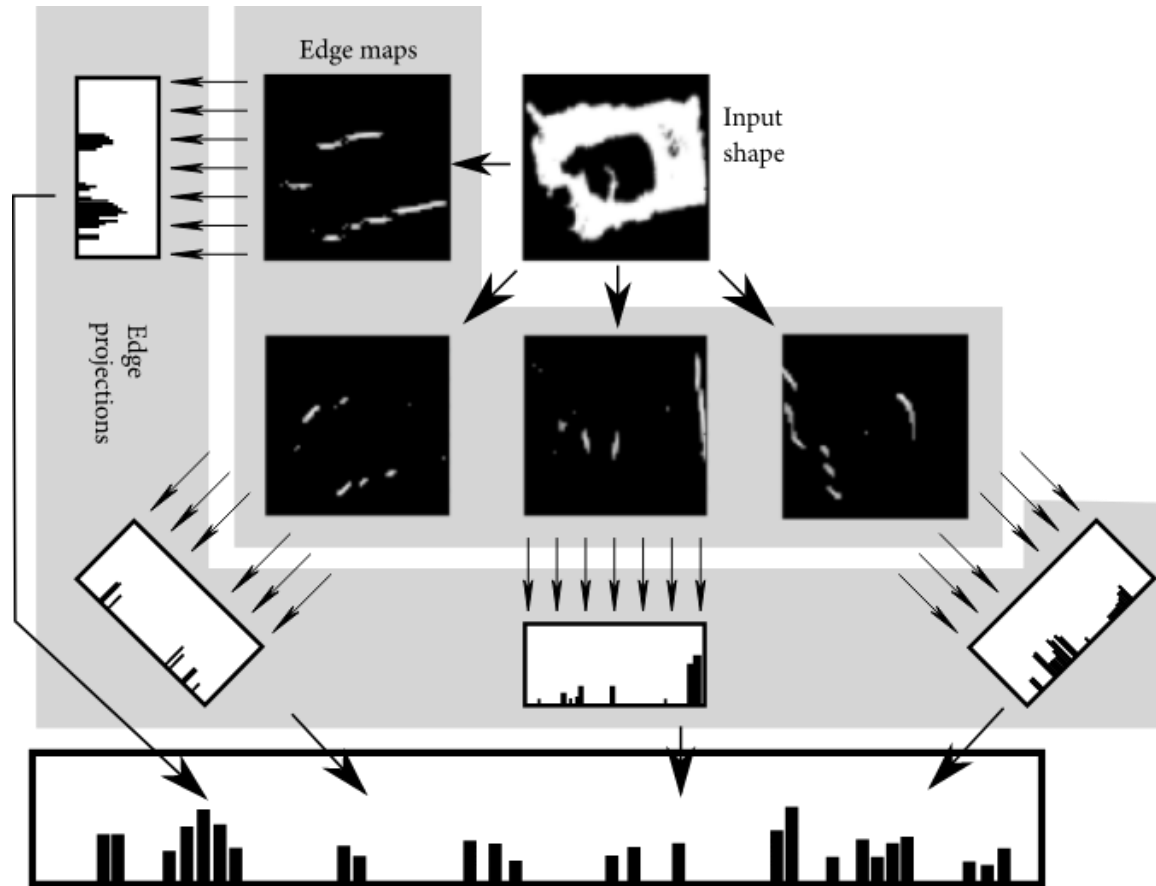


Trích xuất đặc trưng cạnh

- Ta có thể sử dụng các thông tin cạnh trong ảnh để làm vector đặc trưng cho ảnh

Trích xuất đặc trưng cạnh

- Ví dụ



<https://www.hindawi.com/journals/mpe/2018/4763050/>



Trích xuất đặc trưng cạnh

```
def calFeatureVector(img):  
    img_ = cv.resize(img, (256, 256))  
    __, __, G, _ = sobel_filters(img_)  
    row_sum = np.sum(G, axis=1, dtype=np.float64)  
    col_sum = np.sum(G, axis=0, dtype=np.float64)  
    return (np.hstack((row_sum.T, col_sum)))
```





Tài liệu tham khảo

- <https://miac.unibas.ch/SIP/>
- <https://www.mygreatlearning.com/blog/introduction-to-edge-detection/>
- <https://www.slideshare.net/bhavanakhivsara/edge-detection-73118840>
- https://kenndanielso.github.io/mlrefined/blog_posts/14_Convolutional_networks/14_2_Edge_histogram_based_features.html